



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): René Adrián Dávila Pérez

Asignatura: Programación Orientada a Objetos

Grupo: Grupo 7

No de Práctica(s): Práctica 4

Integrante(s): 323064039

426058744

120002850

323071378

323214636

426059576

Semestre: 2027 - 1

Fecha de entrega: 25 de Septiembre de 2026

Observaciones:

CALIFICACIÓN:

Índice

1. Introducción	2
1.1. Objetivos	2
2. Marco Teórico	2
2.1. destructores y recursos	2
2.2. Palabras y frecuencias	2
3. Desarrollo	3
3.1. Investigación sobre destructores	3
3.2. Organización de la aplicación	8
3.3. Algoritmo de conteo	9
4. Resultados	10
4.1. Oración con repeticiones	10
4.2. Oración sin repeticiones	10
5. Conclusiones	12

1. Introducción

A lo largo de esta práctica se estudia la función de un destructor y su relación con el manejo de recursos. Además, se desarrolla una aplicación Java que recibe una oración e imprime cuántas palabras distintas aparecen más de una vez, sin distinguir mayúsculas de minúsculas ni considerar los signos de puntuación como parte de las palabras.

El conteo de palabras permite identificar repeticiones sin revisar manualmente toda una oración. Para obtener resultados consistentes es necesario definir cómo se reconocen las palabras y separar esa tarea de la lectura e impresión. El estudio de los destructores permite distinguir la recuperación de memoria del cierre de otros recursos.

1.1. Objetivos

- Explicar qué hace un destructor y diferenciarlo de la recolección de basura en Java.
- Crear una aplicación que resuelva el conteo de palabras de una oración mediante objetos, ciclos, condicionales y un mapa de frecuencias. La aplicación debe mostrar el total de palabras duplicadas y las frecuencias correspondientes.

2. Marco Teórico

2.1. Destructores y recursos

En C++, un destructor es una función especial de una clase que se ejecuta cuando se destruye un objeto. Puede encargarse de liberar recursos adquiridos por ese objeto, como memoria reservada explícitamente. Para un objeto local de duración automática, la salida de su ámbito provoca su destrucción. El destructor no debe confundirse con cualquier método que borre datos: forma parte del ciclo de vida del objeto.[1]

Java no ofrece destructores como los de C++. Su recolector puede recuperar la memoria de objetos que dejan de ser alcanzables; no se debe suponer que esto ocurre inmediatamente. El antiguo método `finalize()` no proporciona un cierre oportuno y está obsoleto, marcado para eliminación. Los recursos externos requieren mecanismos de cierre explícito; recuperar memoria no equivale a cerrar un archivo.[2]

2.2. Palabras y frecuencias

En esta aplicación, una palabra es una secuencia de letras o dígitos. Los demás caracteres actúan como separadores; así, una coma entre dos palabras no las une. Se normalizan las letras a minúsculas y se conservan los acentos: *Árbol* coincide con

árbol, pero *si* y *sí* permanecen distintos. Se emplean las operaciones de clasificación y conversión de **Character** para los caracteres habituales del español.[3]

Por otro lado, un mapa asocia una clave con un valor. Aquí, la clave es la palabra normalizada y el valor es su número de apariciones. **HashMap** conserva claves únicas sin garantizar un orden de recorrido.[4] Una palabra se considera duplicada cuando su frecuencia es mayor que uno. **TreeSet** permite recorrer claves en su orden natural.[5] **Map** y **Set** describen las operaciones de las colecciones utilizadas, sin requerir clases propias adicionales.

3. Desarrollo

3.1. Investigación sobre destructores

Un ejemplo conceptual es un objeto de C++ que reserva un bloque de memoria para almacenar texto: su destructor puede liberar ese bloque cuando termina la vida del objeto. Esto evita que el recurso quede ocupado después de dejar de utilizarlo.[1] En la aplicación Java de esta práctica, el analizador únicamente conserva una oración y un mapa; no necesita un destructor ni un finalizador.[2]

Al comparar las repuestas de los LLM de cada uno de los integrantes podemos ver que proporcionó respuestas muy similares. Cada LLM ofreció respuestas claras y ejemplos que nos permitieron entender el concepto.

La Figura 3.1 corresponde a la consulta de Luis.

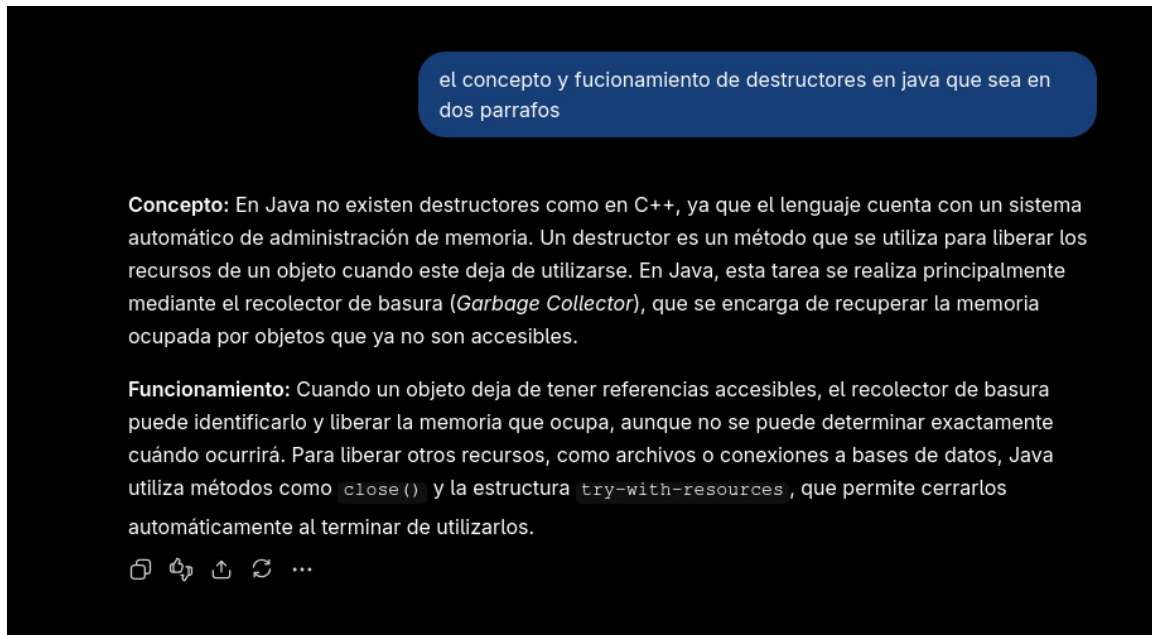


Figura 3.1: Consulta sobre destructores realizada por Luis

La Figura 3.2 corresponde a la consulta de Zoé.

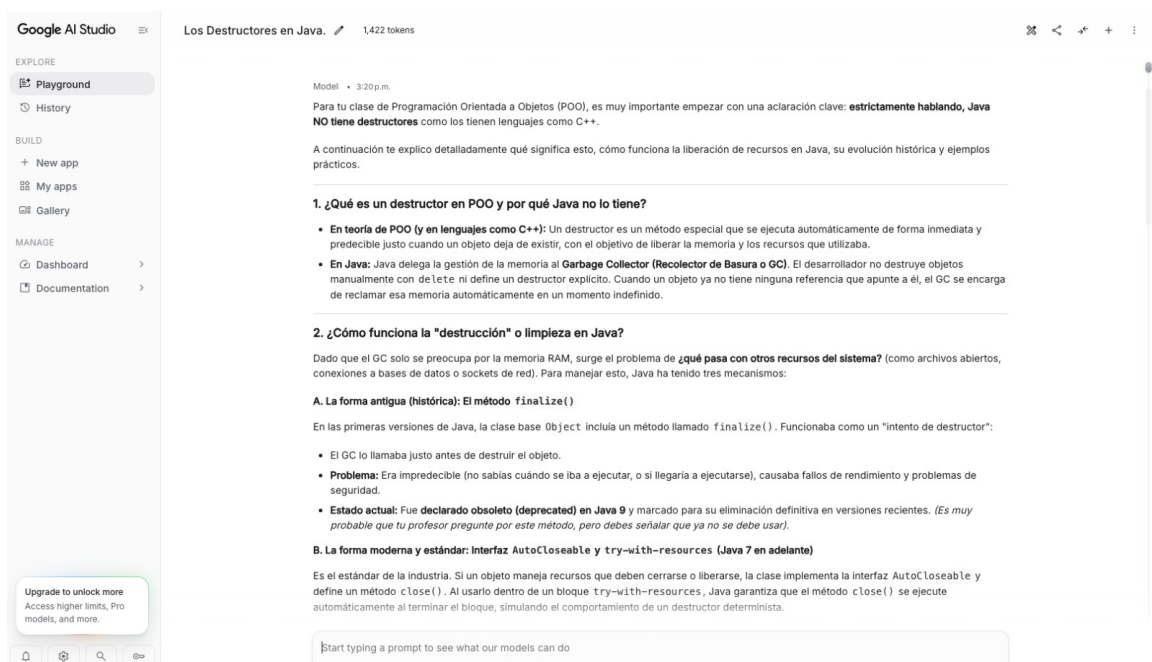


Figura 3.2: Consulta sobre destructores realizada por Zoé

La Figura 3.3 corresponde a la consulta de Alex.

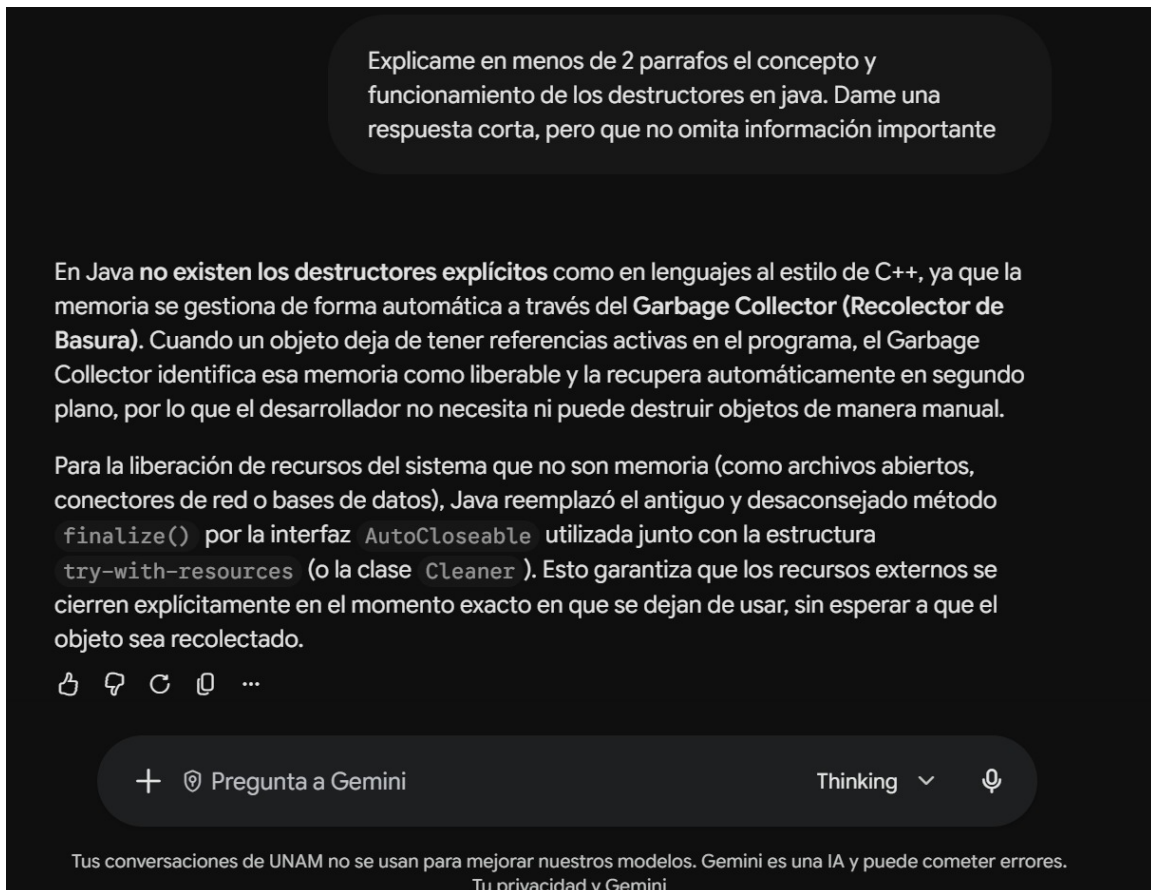


Figura 3.3: Consulta sobre destructores realizada por Alex

La Figura 3.4 corresponde a la consulta de Joseph.

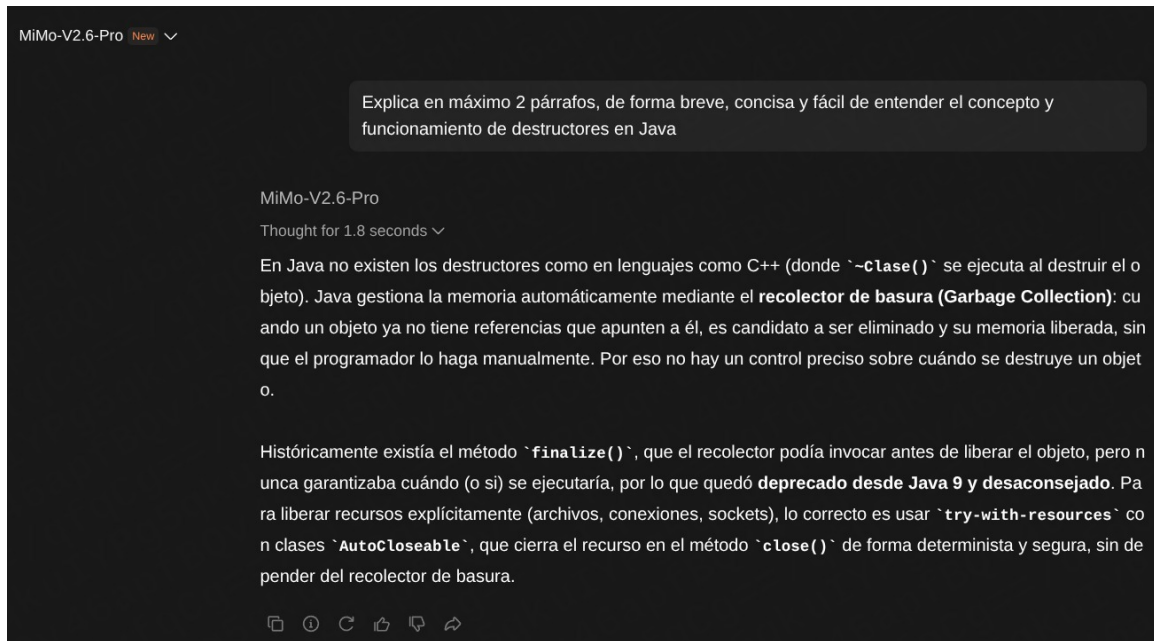


Figura 3.4: Consulta sobre destructores realizada por Joseph

La Figura 3.5 corresponde a la consulta de Hanniel.

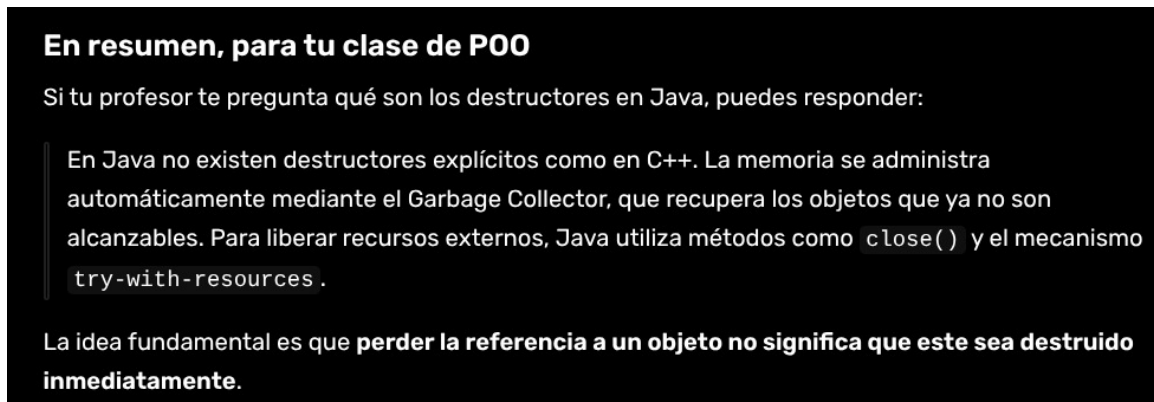


Figura 3.5: Consulta sobre destructores realizada por Hanniel

La Figura 3.6 corresponde a la consulta de Alan.



Figura 3.6: Consulta sobre destructores realizada por Alan

3.2. Organización de la aplicación

`Main` lee una línea con `Scanner`, crea el analizador, solicita el conteo e imprime el total. Después pide mostrar las palabras duplicadas con el orden alfabético correspondiente. Su dependencia de `AnalizadorPalabras` se limita al objeto local que utiliza; no declara atributos ni constructor explícito.

La Figura 3.7 presenta la secuencia de la aplicación.

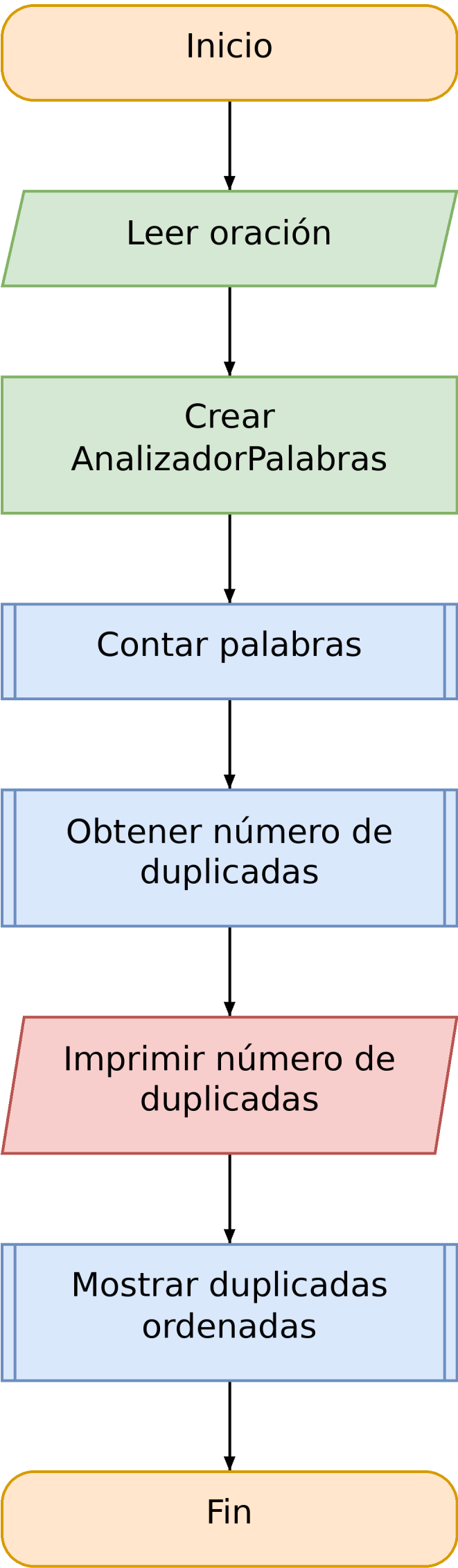


Figura 3.7: Flujo de la clase `Main`

La Figura 3.8 presenta la tabla de Main; la fila de atributos está vacía porque no existen atributos declarados.

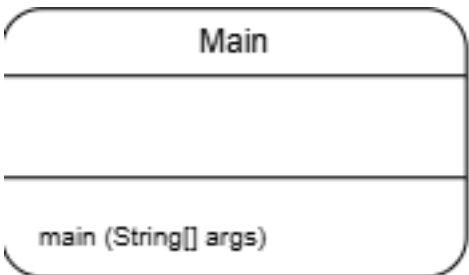


Figura 3.8: Tabla de la clase Main

`AnalizadorPalabras` conserva los atributos públicos `oracion` y `frecuencias`, siguiendo la visibilidad sugerida en el enunciado. Su constructor guarda la oración recibida e inicializa el mapa vacío. `contarPalabras` vacía el mapa antes de comenzar, de modo que repetir el conteo no acumule resultados de una ejecución anterior.

3.3. Algoritmo de conteo

Se recorre la oración carácter por carácter, en cuando se encuentra una letra o un dígito, se agrega su versión en minúscula a una palabra temporal, cuando aparece un separador, se registra la palabra únicamente si no está vacía: si su clave existe, se incrementa la frecuencia; si no existe, se guarda con frecuencia uno, posteriormente se vacía la palabra temporal para comenzar la siguiente.

El recorrido incluye una posición adicional al final, en ella no se lee ningún carácter se ejecuta el mismo cierre usado para un separador, de esta manera se registra la última palabra aunque la oración termine directamente en una letra, los separadores consecutivos no generan claves vacías, sin embargo una línea vacía o formada solo por signos deja el mapa vacío.

Durante el recorrido, el mapa contiene las frecuencias de las palabras completas ya cerradas y la variable temporal contiene únicamente el fragmento de la palabra actual.

La Figura 3.9 resume el conteo, los pasos posteriores de recorrido describen sus operaciones internas condicionales: acumular y registrar corresponden a casos distintos, no a acciones que se ejecuten juntas para cada carácter, el cierre incluye la posición final y solo registra palabras no vacías.



Figura 3.9: Conteo de frecuencias en AnalizadorPalabras

`obtenerNumeroDuplicadas` recorre los valores del mapa y suma una unidad por cada frecuencia mayor que uno.

`mostrarDuplicadas` recorre las claves e imprime únicamente las que cumplen esa condición, si recibe verdadero, utiliza un `TreeSet` local si recibe falso, utiliza el recorrido del mapa sin prometer un orden, consultar o mostrar el resultado no modifica las frecuencias, ya que estos métodos son operaciones independientes que Main invoca después del conteo.

La Figura 3.10 muestra los atributos y las operaciones reales del analizador.

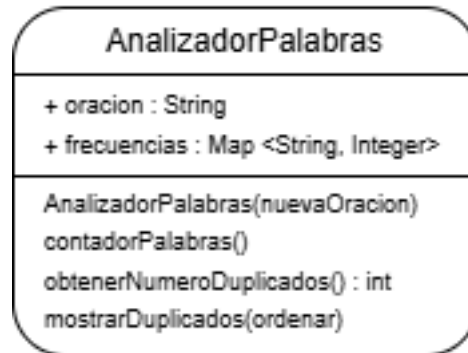


Figura 3.10: Tabla de la clase AnalizadorPalabras

4. Resultados

4.1. Oración con repeticiones

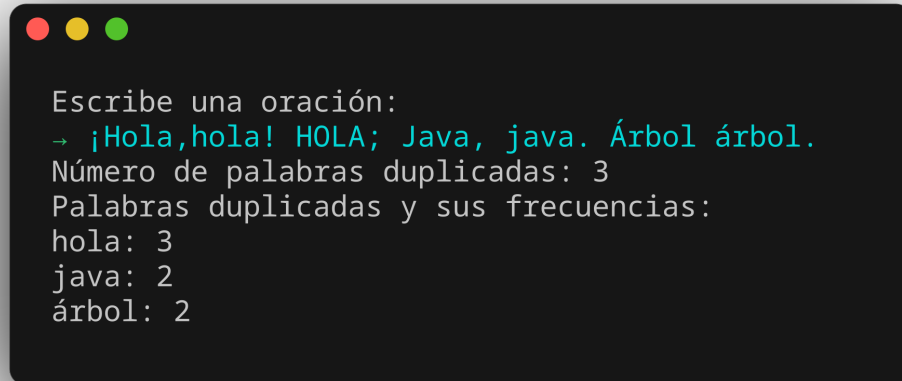
La primera entrada es `¡Hola,hola! HOLA; Java, java. Árbol árbol.` Las frecuencias calculadas son tres para *hola*, dos para *java* y dos para *árbol*. Por lo tanto, el total esperado es de tres palabras duplicadas. El número de apariciones adicionales sería cuatro, pero esa es una medida distinta a la solicitada aquí. Asimismo, la prueba reúne cambios de mayúsculas, acentos, puntuación y una coma sin espacios.

La salida imprime el total tres y después las líneas `hola: 3`, `java: 2` y `árbol: 2`, en ese orden, la Figura 4.1 está destinada a la captura real de esta ejecución.

4.2. Oración sin repeticiones

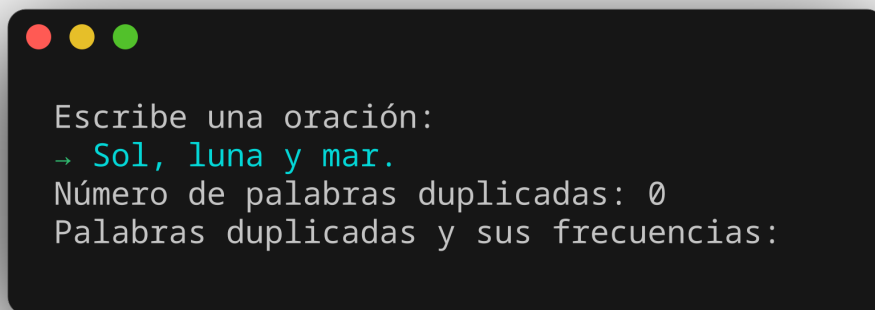
La segunda entrada es `So1, luna y mar.` Cada palabra aparece una vez por lo que se imprime cero y no se agregan líneas de palabras después del encabezado de frecuencias, esta prueba contrasta la existencia y la ausencia de duplicadas, como se puede observar en la Figura 4.2 que es la captura real correspondiente.

Las dos pruebas se verificaron mediante compilación y ejecución Java, comparando la salida completa con los resultados esperados, incluidos los mensajes de lectura y los saltos de línea.



```
Escribe una oración:  
→ ¡Hola,hola! HOLA; Java, java. Árbol árbol.  
Número de palabras duplicadas: 3  
Palabras duplicadas y sus frecuencias:  
hola: 3  
java: 2  
árbol: 2
```

Figura 4.1: Ejecución con tres palabras distintas duplicadas



```
Escribe una oración:  
→ Sol, luna y mar.  
Número de palabras duplicadas: 0  
Palabras duplicadas y sus frecuencias:
```

Figura 4.2: Ejecución sin palabras duplicadas

5. Conclusiones

La implementación realizada permitió validar la importancia del preprocesamiento de texto: la normalización es fundamental para garantizar la precisión, ya que al unificar mayúsculas y gestionar correctamente los signos de puntuación se evita la segmentación artificial de las frecuencias. Asimismo, el uso de mapas de frecuencias demostró ser una estructura eficiente para distinguir entre el conteo de palabras únicas y el volumen total de repeticiones, facilitando el análisis de casos con y sin duplicados.

En cuanto a la arquitectura del código, separar la lógica de interacción en el Main del comportamiento del analizador permitió una mejor organización y modularidad, asegurando una clara correspondencia entre los datos y las operaciones realizadas. Por otro lado, el análisis sobre la gestión de memoria en Java permitió concluir que, a diferencia de otros lenguajes con destructores explícitos, la recolección de basura (Garbage Collector) automatizada hace que no sea recomendable delegar el cierre de recursos externos a los finalizadores.

Referencias

- [1] Microsoft. *Destructors (C++)*. Microsoft Learn. <https://learn.microsoft.com/en-us/cpp/cpp/destructors-cpp>.
- [2] OpenJDK. *JEP 421: Deprecate Finalization for Removal*. <https://openjdk.org/jeps/421>.
- [3] Oracle. *Character*. Java SE 21 API. <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Character.html>.
- [4] Oracle. *HashMap*. Java SE 21 API. <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/HashMap.html>.
- [5] Oracle. *TreeSet*. Java SE 21 API. <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/TreeSet.html>.